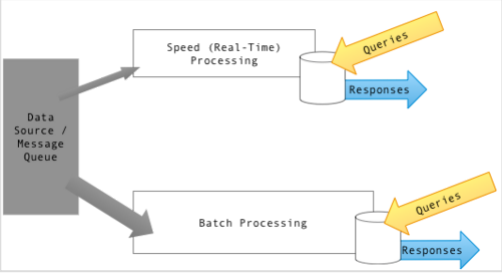
WIKIPEDIA 25
25 years of the free encyclopedia

Lambda architecture

Lambda architecture is a data-processing architecture designed to handle massive quantities of data by taking advantage of both batch and stream-processing methods. This approach to architecture attempts to balance latency, throughput, and fault-tolerance by using batch processing to provide comprehensive and accurate views of batch data, while simultaneously using real-time stream processing to provide views of online data. The two view outputs may be joined before presentation. The rise of lambda architecture is correlated with the growth of big data, real-time analytics, and the drive to mitigate the latencies of map-reduce.[1]



Flow of data through the processing and serving layers of a generic lambda architecture

Lambda architecture depends on a data model with an append-only, immutable data source that serves as a system of record.[2]:32 It is intended for ingesting and processing timestamped events that are appended to existing events rather than overwriting them. State is determined from the natural time-based ordering of the data.

Overview

Lambda architecture describes a system consisting of three layers: batch processing, speed (or real-time) processing, and a serving layer for responding to queries.[3]:13 The processing layers ingest from an immutable master copy of the entire data set. This paradigm was first described by Nathan Marz in a blog post titled "How to beat the CAP theorem" in which he originally termed it the "batch/realtime architecture".[4]

Batch layer

The batch layer precomputes results using a distributed processing system that can handle very large quantities of data. The batch layer aims at perfect accuracy by being able to process *all* available data when generating views. This means it can fix any errors by recomputing based on the complete data set, then updating existing views. Output is typically stored in a read-only database, with updates completely replacing existing precomputed views.[3]:18

By 2014, Apache Hadoop was estimated to be a leading batch-processing system.[5] Later, other, relational databases like Snowflake, Redshift, Synapse and Big Query were also used in this role.

Speed layer

The speed layer processes data streams in real time and without the requirements of fix-ups or completeness. This layer sacrifices throughput as it aims to minimize latency by providing real-time views into the most recent data. Essentially, the speed layer is responsible for filling the "gap" caused by the batch layer's lag in providing views based on the most recent data. This layer's views

may not be as accurate or complete as the ones eventually produced by the batch layer, but they are available almost immediately after data is received, and can be replaced when the batch layer's views for the same data become available.[3]:203

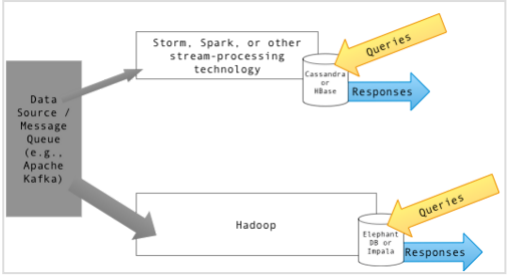


Diagram showing the flow of data through the processing and serving layers of lambda architecture. Example named components are shown.

Stream-processing technologies typically used in this layer include Apache Kafka, Amazon Kinesis, Apache Storm, SQLstream, Apache Samza, Apache Spark, Azure Stream Analytics, Apache Flink. Output is typically stored on fast NoSQL databases.,[6][7] or as a commit log.[8]

Serving layer

Output from the batch and speed layers are stored in the serving layer, which responds to ad-hoc queries by returning precomputed views or building views from the processed data.

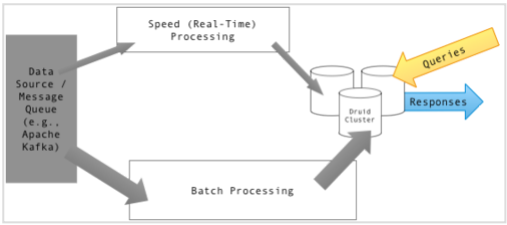


Diagram showing a lambda architecture with a Druid data store.

Examples of technologies used in the serving layer include Apache Druid, Apache Pinot, ClickHouse and Tinybird (https://tinybird.co) which provide a single platform to handle output from both layers.[9] Dedicated stores used in the serving layer include Apache Cassandra, Apache HBase, Azure Cosmos DB, MongoDB, VoltDB or Elasticsearch for speed-layer output, and Elephant DB (https://github.com/nathanmarz/elephantdb), Apache Impala, SAP HANA or Apache Hive for batch-layer output.[2]:45[6]

Optimizations

To optimize the data set and improve query efficiency, various rollup and aggregation techniques are executed on raw data,[9]:23 while estimation techniques are employed to further reduce computation costs.[10] And while expensive full recomputation is required for fault tolerance, incremental computation algorithms may be selectively added to increase efficiency, and techniques such as *partial computation* and resource-usage optimizations can effectively help lower latency.[3]:93,287,293

Lambda architecture in use

Metamarkets, which provides analytics for companies in the programmatic advertising space, employs a version of the lambda architecture that uses Druid for storing and serving both the streamed and batch-processed data.[9]:42

For running analytics on its advertising data warehouse, Yahoo has taken a similar approach, also using Apache Storm, Apache Hadoop, and Druid.[11]:9,16

The Netflix Suro project has separate processing paths for data, but does not strictly follow lambda architecture since the paths may be intended to serve different purposes and not necessarily to provide the same type of views.[12] Nevertheless, the overall idea is to make selected real-time

event data available to queries with very low latency, while the entire data set is also processed via a batch pipeline. The latter is intended for applications that are less sensitive to latency and require a map-reduce type of processing.

Criticism and alternatives

Criticism of lambda architecture has focused on its inherent complexity and its limiting influence. The batch and streaming sides each require a different code base that must be maintained and kept in sync so that processed data produces the same result from both paths. Yet attempting to abstract the code bases into a single framework puts many of the specialized tools in the batch and real-time ecosystems out of reach.[13]

Kappa architecture

Jay Kreps introduced the kappa architecture to use a pure streaming approach with a single code base.[13] In a technical discussion over the merits of employing a pure streaming approach, it was noted that using a flexible streaming framework such as Apache Samza could provide some of the same benefits as batch processing without the latency.[14] Such a streaming framework could allow for collecting and processing arbitrarily large windows of data, accommodate blocking, and handle state.

See also

- Event stream processing

References

1. Schuster, Werner. "Nathan Marz on Storm, Immutability in the Lambda Architecture, Clojure" (http://www.infoq.com/interviews/marz-lambda-architecture). *www.infoq.com*. Interview with Nathan Marz, 6 April 2014
2. Bijnens, Nathan. "A real-time architecture using Hadoop and Storm" (http://lambda-architecture.net/architecture/2013-12-11-a-real-time-architecture-using-hadoop-and-storm-devoxx). 11 December 2013.
3. Marz, Nathan; Warren, James. *Big Data: Principles and best practices of scalable realtime data systems*. Manning Publications, 2013.
4. Marz, Nathan. "How to beat the CAP theorem" (http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html). 13 October 2011.
5. Kar, Saroj. "Hadoop Sector will Have Annual Growth of 58% for 2013-2020" (http://cloudtimes.org/2014/05/28/hadoop-sector-will-have-annual-growth-of-58-for-2013-2020/) Deprecated link archived 2014-08-26 at archive.today, 28 May 2014. *Cloud Times*.
6. Kinley, James. "The Lambda architecture: principles for architecting realtime Big Data systems" (https://jameskinley.tumblr.com/post/37398560534/the-lambda-architecture-principles-for-architecting) Archived (https://web.archive.org/web/20140904183723/http://jameskinley.tumblr.com/post/37398560534/the-lambda-architecture-principles-for-architecting) 2014-09-04 at the Wayback Machine, retrieved 26 August 2014.
7. Ferrera Bertran, Pere. "Lambda Architecture: A state-of-the-art" (https://web.archive.org/web/20190312082929/http://www.datasalt.com/2014/01/lambda-architecture-a-state-of-the-art/). 17 January 2014, Datasalt.

8. Confluent."Kafka and Events – Key/Value Pairs" (https://developer.confluent.io/what-is-apache-kafka/#kafka-and-events--keyvalue-pairs), retrieved 06 October 2022.
9. Yang, Fangjin, and Merlino, Gian. "Real-time Analytics with Open Source Technologies" (https://speakerdeck.com/druidio/real-time-analytics-with-open-source-technologies-1). 30 July 2014.
10. Ray, Nelson. "The Art of Approximating Distributions: Histograms and Quantiles at Scale" (https://metamarkets.com/2013/histograms/). 12 September 2013. Metamarkets.
11. Rao, Supreeth; Gupta, Sunil. "Interactive Analytics in Human Time" (https://www.slideshare.net/Hadoop_Summit/interactive-analytics-in-human-time?next_slideshow=1). 17 June 2014
12. Bae, Jae Hyeon; Yuan, Danny; Tonse, Sudhir. "Announcing Suro: Backbone of Netflix's Data Pipeline" (http://techblog.netflix.com/2013/12/announcing-suro-backbone-of-netflixs.html), *Netflix*, 9 December 2013
13. Kreps, Jay (2 July 2014). "Questioning the Lambda Architecture" (https://www.oreilly.com/radar/questioning-the-lambda-architecture/). *oreilly.com*. Oreilly. Retrieved 3 October 2024.
14. Hacker News (https://news.ycombinator.com/item?id=7976785) retrieved 20 August 2014

Retrieved from "https://en.wikipedia.org/w/index.php?title=Lambda_architecture&oldid=1275106656"